Course Title: Cryptography and Network Security Lab	**Course Code**: BCSE309P
Faculty: Prof. Karthika V	Slot: *L15+L16*
Regno: *23BCE1099*	Name: *Arkita Barua*

Ex2a

S-DES

Code

Sender

```
import java.net.*;
import java.io.*;
import java.util.*;

public class sdes_client_sender {

    // Permutation tables
    static int[] P10 = {3,5,2,7,4,10,1,9,8,6};
    static int[] P8  = {6,3,7,4,8,5,10,9};
    static int[] IP  = {2,6,3,1,4,8,5,7};
    static int[] IP_INV = {4,1,3,5,7,2,8,6};
    static int[] EP = {4,1,2,3,2,3,4,1};
    static int[] P4 = {2,4,3,1};

    static int[][] S0 = {
        {1,0,3,2},
        {3,2,1,0},
        {0,2,1,3},
        {3,1,3,2}
    };

    static int[][] S1 = {
        {0,1,2,3},
        {2,0,1,3},
        {3,0,1,0},
        {2,1,0,3}
    };

    public static void main(String[] args) throws Exception {
```

```java
        Scanner sc = new Scanner(System.in);

        Socket socket = new Socket("localhost", 5000);
        PrintWriter out = new PrintWriter(socket.getOutputStream(), true);

        System.out.print("Plaintext (8 bits): ");
        String pt = sc.nextLine();

        System.out.print("Key (10 bits): ");
        String key = sc.nextLine();

        // Generate Subkeys
        String p10 = permute(key, P10);
        String left = p10.substring(0,5);
        String right = p10.substring(5);

        left = shift(left,1);
        right = shift(right,1);

        String k1 = permute(left+right, P8);

        left = shift(left,2);
        right = shift(right,2);

        String k2 = permute(left+right, P8);

        System.out.println("\nSubkeys:");
        System.out.println("K1: " + k1);
        System.out.println("K2: " + k2);

        // Initial Permutation
        String ip = permute(pt, IP);
        System.out.println("IP: " + ip);

        // Round 1
        String r1 = fk(ip, k1);
        System.out.println("Round1: " + r1);

        // Swap
        r1 = r1.substring(4) + r1.substring(0,4);

        // Round 2
        String r2 = fk(r1, k2);
        System.out.println("Round2: " + r2);

        // Inverse IP
        String ct = permute(r2, IP_INV);
```

```java
        System.out.println("Cipher Text: " + ct);

        // Send to receiver
        out.println(ct);
        out.println(k1);
        out.println(k2);

        socket.close();
        sc.close();
    }

    static String permute(String input, int[] table) {
        String output = "";
        for(int i : table)
            output += input.charAt(i-1);
        return output;
    }

    static String shift(String input, int n) {
        return input.substring(n) + input.substring(0,n);
    }

    static String xor(String a, String b) {
        String result = "";
        for(int i=0;i<a.length();i++)
            result += (a.charAt(i)==b.charAt(i)) ? '0' : '1';
        return result;
    }

    static String fk(String input, String key) {
        String left = input.substring(0,4);
        String right = input.substring(4);

        String ep = permute(right, EP);
        String xor = xor(ep, key);

        String s0 = sBox(xor.substring(0,4), S0);
        String s1 = sBox(xor.substring(4), S1);

        String p4 = permute(s0+s1, P4);
        String result = xor(left, p4) + right;

        return result;
    }

    static String sBox(String input, int[][] box) {
        int row = Integer.parseInt("" + input.charAt(0) + input.charAt(3),2);
        int col = Integer.parseInt("" + input.charAt(1) + input.charAt(2),2);
```

```
        int val = box[row][col];
        return String.format("%2s", Integer.toBinaryString(val)).replace('
','0');
    }
}
```

Receiver

```
import java.net.*;
import java.io.*;

public class sdes_server_receiver {

    // Same permutation tables
    static int[] IP  = {2,6,3,1,4,8,5,7};
    static int[] IP_INV = {4,1,3,5,7,2,8,6};
    static int[] EP = {4,1,2,3,2,3,4,1};
    static int[] P4 = {2,4,3,1};

    static int[][] S0 = {
        {1,0,3,2},
        {3,2,1,0},
        {0,2,1,3},
        {3,1,3,2}
    };

    static int[][] S1 = {
        {0,1,2,3},
        {2,0,1,3},
        {3,0,1,0},
        {2,1,0,3}
    };

    public static void main(String[] args) throws Exception {

        ServerSocket serverSocket = new ServerSocket(5000);
        System.out.println("Waiting for sender(client)");

        Socket socket = serverSocket.accept();
        System.out.println("client connected");

        BufferedReader in = new BufferedReader(new
InputStreamReader(socket.getInputStream()));

        String ct = in.readLine();
        String k1 = in.readLine();
```

```java
        String k2 = in.readLine();

        System.out.println("\nReceived Cipher Text: " + ct);
        System.out.println("Subkeys:");
        System.out.println("K1: " + k1);
        System.out.println("K2: " + k2);

        String ip = permute(ct, IP);
        System.out.println("IP: " + ip);

        String r1 = fk(ip, k2);
        System.out.println("Round1: " + r1);

        r1 = r1.substring(4) + r1.substring(0,4);

        String r2 = fk(r1, k1);
        System.out.println("Round2: " + r2);

        String pt = permute(r2, IP_INV);
        System.out.println("Plaintext: " + pt);

        socket.close();
        serverSocket.close();
    }


    static String permute(String input, int[] table) {
        String output = "";
        for(int i : table)
            output += input.charAt(i-1);
        return output;
    }

    static String xor(String a, String b) {
        String result = "";
        for(int i=0;i<a.length();i++)
            result += (a.charAt(i)==b.charAt(i)) ? '0' : '1';
        return result;
    }

    static String fk(String input, String key) {
        String left = input.substring(0,4);
        String right = input.substring(4);

        String ep = permute(right, EP);
        String xor = xor(ep, key);

        String s0 = sBox(xor.substring(0,4), S0);
```

```java
        String s1 = sBox(xor.substring(4), S1);

        String p4 = permute(s0+s1, P4);
        String result = xor(left, p4) + right;

        return result;
    }

    static String sBox(String input, int[][] box) {
        int row = Integer.parseInt("" + input.charAt(0) + input.charAt(3),2);
        int col = Integer.parseInt("" + input.charAt(1) + input.charAt(2),2);
        int val = box[row][col];
        return String.format("%2s", Integer.toBinaryString(val)).replace(' ','0');
    }
}
```

Output [screenshot]

Sender

```
PS E:\aa vit sem6\crypto-lab> javac sdes_client_sender.java
PS E:\aa vit sem6\crypto-lab> java sdes_client_sender
Plaintext (8 bits): 00101000
Key (10 bits): 1100011110

Subkeys:
K1: 11101001
K2: 10100111
IP: 00100010
Round1: 00110010
Round2: 00010011
Cipher Text: 10001010
PS E:\aa vit sem6\crypto-lab>
```

Receiver

```
PS E:\aa vit sem6\crypto-lab> java sdes_server_receiver
Waiting for sender(client)
client connected

Received Cipher Text: 10001010
Subkeys:
K1: 11101001
K2: 10100111
IP: 00010011
Round1: 00100011
Round2: 00100010
Plaintext: 00101000
PS E:\aa vit sem6\crypto-lab>
```